

LAPORAN PRAKTIKUM
PRAKTIKUM PEMROGRAMAN WEB 1
PERTEMUAN 10
JAVASCRIPT



Disusun oleh:

Nama : Fahmi Hasan
NIM : 25/566255/SV/27049
Kelas : PL2B1
Dosen Pengampu : Achmad Choirudin Emcha, S.Kom., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

1. Tujuan Percobaan.....	3
2. Dasar Teori.....	3
2.1 Javascript.....	3
2.2 Metode Integrasi.....	3
2.3 Deklarasi Variabel.....	4
2.4 Operator dan Alur Kontrol (Control Flow).....	4
2.5 Manipulasi Document Object Model (DOM).....	4
3. Hasil dan Pembahasan	5
3.1 Penjelasan Implementasi Kode Mini Project	5
3.2 Tugas 1	7
3.3 Tugas 2	9
4. Kesimpulan	12
5. Daftar Pustaka.....	12

PERTEMUAN 10

JAVASCRIPT

1. Tujuan Percobaan

1. Mahasiswa mampu memahami dasar penggunaan JavaScript
2. Mahasiswa mampu mengimplementasikan tiga metode integrasi
3. Mahasiswa mampu merancang dan menerapkan logika kontrol program.
4. Mahasiswa mampu melakukan manipulasi Document Object Model (DOM).

2. Dasar Teori

2.1 Javascript

JavaScript adalah bahasa pemrograman tingkat tinggi yang pada awalnya dikembangkan pada tahun 1995 dengan nama Mocha, sebelum akhirnya dikenal luas sebagai JavaScript. Bahasa ini pada dasarnya dirancang untuk dieksekusi oleh peramban web (browser) guna menciptakan interaktivitas pada halaman web. Seiring dengan perkembangan teknologi, ekosistem JavaScript meluas hingga memungkinkan eksekusi di luar peramban melalui lingkungan runtime seperti Node.js. Sifat utama dari bahasa ini adalah dynamic typing, di mana tipe data dari sebuah variabel ditentukan secara otomatis pada saat eksekusi program (runtime) berdasarkan nilai yang diberikan, sehingga pengembang tidak perlu mendeklarasikan tipe data secara eksplisit.

2.2 Metode Integrasi

Penyisipan kode External JavaScript ke dalam dokumen HTML dapat dilakukan melalui tiga pendekatan utama:

- Embed
Menuliskan kode langsung di dalam tag `<script>`, yang dapat diletakkan di bagian `<head>` atau di akhir `<body>`. Penempatan di akhir `<body>`.
- Inline
Menyematkan kode langsung ke dalam atribut HTML yang bertindak sebagai pemicu peristiwa (event trigger), seperti atribut `onclick` pada elemen tombol atau tautan.

Memisahkan kode ke dalam berkas berekstensi `.js`. Metode ini merupakan standar untuk proyek berskala besar karena mendukung modularitas dan memudahkan pemeliharaan basis kode.

Untuk pengujian dan interaksi pengguna, JavaScript menyediakan beberapa metode output, antara lain `console.log()` untuk penelusuran masalah (debugging), `alert()` untuk menampilkan kotak dialog peringatan, `document.write()` untuk penulisan langsung ke dokumen, dan manipulasi properti `innerHTML` untuk memperbarui konten elemen spesifik.

2.3 Deklarasi Variabel

Standar modern JavaScript memperkenalkan pembaruan pendeklarasian variabel untuk menghindari masalah yang sering terjadi pada standar lama.

- Var
Merupakan metode deklarasi lama yang memiliki cakupan fungsi (function scope). Penggunaan var rentan terhadap deklarasi ulang yang tidak disengaja dan kini mulai dihindari dalam praktik pengembangan modern.
- Let
Digunakan untuk mendeklarasikan variabel yang nilainya dapat diubah (mutable). Pendeklarasian ini menggunakan cakupan blok (block scope), yang berarti variabel hanya dapat diakses di dalam blok tempat ia dideklarasikan.
- Const
Digunakan untuk nilai yang bersifat tetap atau konstan (immutable). Sama seperti let, const juga menganut block scope dan merupakan pilihan utama secara asali (default) untuk mencegah mutasi data yang tidak disengaja.

2.4 Operator dan Alur Kontrol (Control Flow)

JavaScript mendukung berbagai jenis operator untuk memanipulasi data, meliputi operator aritmatika, penugasan, logika (&&, ||, !), dan bitwise. Salah satu aspek kritis dalam bahasa ini adalah penggunaan operator perbandingan. Pengembang sangat disarankan untuk menggunakan perbandingan identitas ketat atau strict equality (=== dan !==) dibandingkan perbandingan longgar atau loose equality (== dan !=). Operator ketat tidak hanya mengevaluasi kesamaan nilai, tetapi juga memeriksa kecocokan tipe data secara presisi, sehingga meminimalisasi anomali konversi tipe data otomatis.

2.5 Manipulasi Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman dari dokumen HTML dalam bentuk pohon objek (object tree). Melalui JavaScript, pengembang dapat menyeleksi elemen DOM menggunakan metode seperti getElementById atau querySelector, lalu melakukan mutasi terhadap struktur dokumen tersebut tanpa perlu memuat ulang halaman (reload). Mutasi ini dapat berupa pengubahan konten teks (textContent), manipulasi kelas CSS (classList), hingga pembuatan dan penghapusan simpul elemen baru (createElement dan remove).

Responsivitas halaman terhadap pengguna dikendalikan melalui event listener, yaitu metode terstruktur untuk mendeteksi tindakan pengguna seperti klik tetikus (mouse click) atau ketikan papan tik (keyboard input), yang kemudian direspons dengan mengeksekusi blok kode tertentu guna menciptakan halaman web yang dinamis.

3. Hasil dan Pembahasan

3.1 Penjelasan Implementasi Kode Mini Project

Implementasi Kode

```
1 <!doctype html>
2 <html lang="id">
3 <head>
4 <meta charset="UTF-8" />
5 <title>To-Do List</title>
6 <link
7 href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
8 rel="stylesheet"
9 />
10
11 </head>
12 <body class="bg-light">
13 <div class="container mt-5" style="max-width: 500px">
14 <h2 class="mb-4"> To-Do List</h2>
15 <div class="input-group mb-3">
16 <input
17 type="text"
18 id="inputTugas"
19 class="form-control"
20 placeholder="Tambah tugas baru..."
21 />
22 <button class="btn btn-primary" id="btnTambah">Tambah</button>
23 </div>
24 <ul class="list-group" id="daftarTugas"></ul>
25 <p class="text-muted mt-2" id="info">Tidak ada tugas</p>
26 </div>
27
28 <script>
29 const input = document.getElementById("inputTugas");
30 const btnTambah = document.getElementById("btnTambah");
31 const daftar = document.getElementById("daftarTugas");
32 const info = document.getElementById("info");
33 let jumlah = 0;
34
35 function perbaruiInfo() {
36   info.textContent =
37     jumlah === 0 ? "Tidak ada tugas" : jumlah + " tugas tersisa";
38 }
39
40 function tambahTugas() {
41   const teks = input.value.trim();
42   if (teks === "") {
43     alert("Tugas tidak boleh kosong!");
44     return;
45   }
46   const li = document.createElement("li");
47   li.className =
48     "list-group-item d-flex justify-content-between align-items-center";
49   const span = document.createElement("span");
50   span.textContent = teks;
51   span.style.cursor = "pointer";
52   span.title = "Klik untuk tandai selesai";
53   span.addEventListener("click", () => {
54     const done = span.style.textDecoration === "line-through";
55     span.style.textDecoration = done ? "none" : "line-through";
56     span.style.opacity = done ? "1" : "0.5";
57   });
58
59   const btnHapus = document.createElement("button");
60   btnHapus.textContent = "x";
61   btnHapus.className = "btn btn-sm btn-outline-danger";
62   btnHapus.addEventListener("click", () => {
63     li.remove();
64     jumlah--;
65     perbaruiInfo();
66   });
67
68   li.appendChild(span);
69   li.appendChild(btnHapus);
70   daftar.appendChild(li);
71   input.value = "";
72   input.focus();
73   jumlah++;
74   perbaruiInfo();
75 }
76
77 btnTambah.addEventListener("click", tambahTugas);
78 input.addEventListener("keydown", (e) => {
79   if (e.key === "Enter") tambahTugas();
80 });
81 </script>
82 </body>
83 </html>
84
```

Analisis Kode

Proyek mini To-Do List ini menggabungkan HTML dan JavaScript melalui manipulasi DOM. Pertama, elemen-elemen antarmuka seperti kolom input, tombol, dan daftar tugas () dipilih menggunakan `getElementById` dan disimpan dalam variabel `const`. Untuk melacak jumlah tugas, digunakan variabel `let jumlah` yang nilainya terus diperbarui. Teks informasi terkait sisa tugas diatur oleh fungsi `perbaruiInfo()`, yang memanfaatkan `textContent` dan operator ternary untuk menampilkan teks "Tidak ada tugas" saat kosong, atau angka sisa tugas jika ada.

Inti program berada pada fungsi `tambahTugas()`. Saat dijalankan, input divalidasi agar tidak kosong, lalu program membuat elemen daftar () baru di memori menggunakan `createElement`. Di dalam tersebut disisipkan teks tugas yang dilengkapi Event Listener; jika diklik, teks akan tercoret (line-through) sebagai penanda selesai. Terdapat juga tombol hapus dinamis yang, saat diklik, akan membuang elemen tersebut dari DOM dengan perintah `.remove()` dan mengurangi hitungan tugas. Semua elemen ini kemudian dirangkai dengan `.appendChild()` dan dimasukkan ke dalam daftar utama.

Agar aplikasi lebih praktis digunakan, fungsi `tambahTugas()` dihubungkan dengan dua Event Listener utama. Pengguna dapat menambahkan daftar pekerjaannya dengan cara mengeklik tombol "Tambah", atau dengan menekan tombol "Enter" pada keyboard saat mengetik di kolom input. Deteksi tombol keyboard ini dilakukan dengan menangkap event `keydown` dan menggunakan percabangan `if` untuk memastikan bahwa tombol yang ditekan benar-benar "Enter".

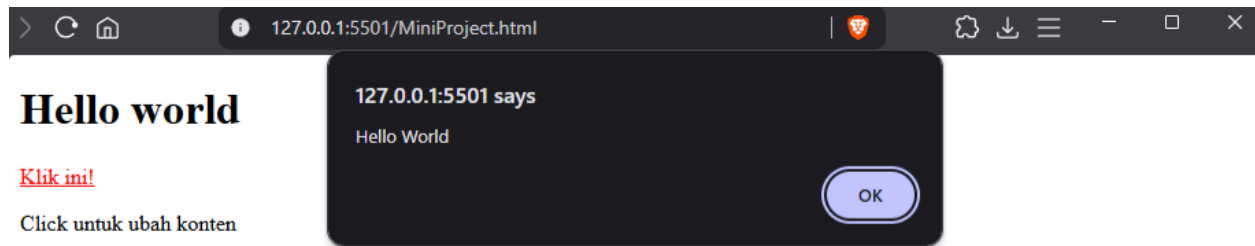
3.2 Tugas 1

Implementasi Kode

```
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Tugas 1</title>
8  </head>
9
10 <body>
11     <h1>Hello world</h1>
12
13     <!-- Penulisan Inline Menggunakan Alert -->
14     <a href="#" onclick="alert('Hello World')">Klik ini!</a>
15
16     <!-- Penulisan Eksternal File Menggunakan Document Write -->
17     <script src="tugas1.js"></script>
18
19     <!-- Penulisan Embed Menggunakan Output InnerHTML -->
20     <p id="change" onclick="changeContent()">Click untuk ubah konten</p>
21
22     <script>
23         //Penulisan Embed Menggunakan Output console.log
24         console.log('Helo JS dari body');
25
26         function changeContent() {
27             document.getElementById("change").innerHTML = 'Hello world lagi'
28         };
29     </script>
30 </body>
31
32 </html>
```

```
1  document.write("Ini Dari Eksternal File")
```

Hasil Tampilan



Analisis Kode

Kode ini mempraktikkan tiga metode utama untuk menulis dan memanggil JavaScript:

- Inline JavaScript (`onclick="alert(...)"`)
Pada elemen `<a href="#" onclick="alert('Hello World')">`, JavaScript ditulis langsung di dalam atribut HTML. Metode ini cocok untuk eksekusi kode sederhana.
- External JavaScript (`<script src="tugas1.js"></script>`)
Kode ini memanggil file terpisah bernama `tugas1.js`. Ini adalah cara terbaik dalam pemrograman web karena memisahkan file HTML dan JavaScript, membuat kode lebih rapi, mudah dikelola, dan dikembangkan.
- Embedded / Internal JavaScript (`<script>...</script>`)
Kode JavaScript ditulis di dalam tag `<script>` yang disisipkan langsung ke dalam `<body>`. Ini berguna untuk skrip yang hanya spesifik berjalan di halaman tersebut saja.

Metode Output JavaScript

Kode ini juga mendemonstrasikan cara JavaScript memberikan umpan balik (output) kepada pengguna atau developer:

- `alert()`: Memunculkan kotak pop-up pesan.
- `innerHTML`: Mengubah teks atau konten HTML secara langsung saat elemen diklik.
- `console.log()`: Mencetak pesan tersembunyi di Developer Tools browser untuk keperluan debugging.
- `document.write()`: (Diasumsikan ada di file `tugas1.js`) untuk menulis teks langsung ke dalam dokumen HTML.

3.3 Tugas 2

Implementasi Kode

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Tugas2</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
8 </head>
9
10 <body class="bg-light">
11   <div class="container mt-5">
12     <h2 class="text-center mb-4">Sistem Pendataan Praktikan PPM1</h2>
13
14     <div id="hasil-output">
15       <p class="text-center text-muted">Menunggu konfirmasi data...</p>
16     </div>
17   </div>
18
19   <script>
20     // Menggunakan confirm() untuk konfirmasi Ya/Tidak
21     let isMahasiswa = confirm("Apakah Anda mahasiswa PPM1?");
22
23     // Memilih elemen target untuk menampilkan output
24     let hasilDiv = document.getElementById("hasil-output");
25
26     if (isMahasiswa) {
27       // Jika memilih OK (true), gunakan prompt untuk meminta input
28       let nama = prompt("Masukkan Nama Anda:", "");
29       let nim = prompt("Masukkan NIM Anda:", "");
30       let angkatan = prompt("Masukkan Tahun Angkatan (misal: 2024):", "");
31
32       // Pastikan user tidak membatalkan prompt (null)
33       if (nama !== null && nim !== null && angkatan !== null) {
34
35         // 1. Menghitung sisa tahun kelulusan dengan asumsi 4 tahun dari angkatan
36         let tahunAngkatan = parseInt(angkatan);
37         let tahunSaatIni = 2024;
38         let tahunLulus = tahunAngkatan + 4;
39         let sisaTahun = tahunLulus - tahunSaatIni;
40
41         // Menggunakan operator ternary untuk menampilkan teks yang sesuai
42         let teksLulus = sisaTahun > 0 ? `${sisaTahun} tahun lagi` : "Kamu sudah lulus (seharusnya)";
43
44         // 2. Mengecek NIM Genap atau Ganjil dengan validasi angka
45         let angkaNim = parseInt(nim);
46         let statusNim;
47
48         if (isNaN(angkaNim)) {
49           statusNim = "Input Bukan Angka!"; // Jika yang dimasukkan huruf
50         } else {
51           statusNim = (angkaNim % 2 === 0) ? "Genap" : "Ganjil"; // Jika benar angka
52         }
53
54         // Menampilkan hasil dalam tabel Bootstrap menggunakan innerHTML
55         hasilDiv.innerHTML = `
56           <div class="card shadow-sm">
57             <div class="card-body">
58               <h5 class="card-title text-success">Data Berhasil Disimpan!</h5>
59               <div class="table-responsive mt-3">
60                 <table class="table table-bordered table-hover">
61                   <thead class="table-dark">
62                     <tr>
63                       <th>Nama</th>
64                       <th>NIM</th>
65                       <th>Status NIM</th>
66                       <th>Angkatan</th>
67                       <th>Estimasi Lulus</th>
68                     </tr>
69                   </thead>
70                   <tbody>
71                     <tr>
72                       <td>${nama}</td>
73                       <td>${nim}</td>
74                       <td><span class="badge bg-primary">${statusNim}</span></td>
75                       <td>${angkatan}</td>
76                       <td>${teksLulus}</td>
77                     </tr>
78                   </tbody>
79                 </table>
80               </div>
81             </div>
82           </div>
83           `;
84         } else {
85           hasilDiv.innerHTML = `<div class="alert alert-warning">Proses pendataan dibatalkan.</div>`;
86         }
87       } else {
88         // Jika TIDAK (Cancel), tampilkan pesan penolakan
89         hasilDiv.innerHTML = `
90           <div class="alert alert-danger text-center shadow-sm p-4">
91             <h4>Akses Ditolak! </h4>
92             <p class="mb-0">Maaf, web ini khusus untuk para mahasiswa Praktikum Pemrograman Web 1. Silakan kembali jika anda sudah menjadi mahasiswa Praktikum Pemrograman Web 1!</p>
93           </div>
94           `;
95       }
96     }
97   </script>
98 </body>
99 </html>
```

Hasil Tampilan



Sistem Pendataan Praktikan PPW1

Data Berhasil Disimpan!

Nama	NIM	Status NIM	Angkatan	Estimasi Lulus
Fahmi Hasan	266255	Ganjil	2025	5 tahun lagi



Sistem Pendataan Praktikan PPW1

Akses Ditolak!

Maaf, web ini khusus untuk para mahasiswa Praktikum Pemrograman Web 1. Silakan kembali jika anda sudah menjadi mahasiswa Praktikum Pemrograman Web 1!



Sistem Pendataan Praktikan PPW1

Proses pendataan dibatalkan.

Analisis Kode

Interaksi Pengguna (Browser Dialogs)

Kode menggunakan tiga jenis fungsi antarmuka bawaan browser untuk berinteraksi dengan pengguna:

- `confirm()`: Digunakan di awal untuk memvalidasi apakah pengguna adalah Mahasiswa Praktikum Perrograman Web 1. Fungsi ini mengembalikan nilai boolean (`true` jika "OK", `false` jika "Cancel").
- `prompt()`: Jika pengguna memilih "OK", program memunculkan tiga kotak input berurutan untuk meminta data Nama, NIM, dan Angkatan.
- Validasi Batal (`!== null`): Program mengecek apakah pengguna menekan tombol "Cancel" saat mengisi `prompt`. Jika dibatalkan, proses dihentikan dan memunculkan notifikasi peringatan.

Logika Pemrosesan Data

Setelah data terkumpul, JavaScript melakukan beberapa pemrosesan logika dasar:

- Konversi Tipe Data: Menggunakan `parseInt()` untuk mengubah input angkatan dan nim (yang pada awalnya berupa string dari `prompt`) menjadi angka (integer).
- Kondisi Ganjil/Genap: Mengecek apakah NIM berupa angka. Jika valid, digunakan operator modulo (`% 2 === 0`) bersama operator ternary untuk menentukan apakah NIM tersebut Ganjil atau Genap.
- Estimasi Kelulusan: Menghitung selisih tahun kelulusan (`angkatan + 4`) dengan tahun saat ini dengan menggunakan operator ternary.

Manipulasi DOM dan Tampilan (UI)

- Penggunaan Bootstrap 5: Kode ini menggunakan framework CSS Bootstrap (melalui CDN) untuk membuat tampilan.
- Output Dinamis (`innerHTML`): Program menyuntikkan elemen HTML ke dalam `<div id="hasil-output">` berdasarkan kondisi yang terpenuhi (Tabel Data, Pesan Batal, atau Pesan Akses Ditolak).

4. Kesimpulan

Berdasarkan implementasi kode dan analisis hasil praktikum yang telah dilakukan, dapat ditarik kesimpulan bahwa JavaScript memiliki peran krusial dalam mengubah halaman web statis menjadi dinamis dan interaktif. Selain itu tujuan praktikum yang telah ditetapkan pada tujuan praktikum berhasil dicapai dengan penjabaran sebagai berikut:

- Penerapan Metode Integrasi: Mahasiswa berhasil mengimplementasikan tiga metode penyisipan JavaScript pada dokumen HTML. Penggunaan metode external terbukti menjadi standar praktik terbaik untuk menjaga agar kode HTML tetap bersih dan modular, sedangkan metode embed dan inline dapat difungsikan untuk pengujian skala kecil atau pemicu peristiwa spesifik.
- Implementasi Alur Kontrol: Penerapan logika pemrograman berbasis JavaScript telah berhasil dilakukan melalui manipulasi variabel yang secara efektif mencegah kebocoran cakupan blok (block scope leak).
- Manipulasi DOM dan Interaktivitas Antarmuka: Mahasiswa mampu mengimplementasikan bagaimana JavaScript mengontrol Document Object Model (DOM). Melalui metode seleksi elemen dan implementasi event listener, halaman web dapat merespons berbagai interaksi dengan pengguna seperti pengisian formulir, penekanan tombol tetikus, dan keyboard untuk memasukkan input.

5. Daftar Pustaka

[1] Mozilla Developer Network (MDN), "JavaScript: Dynamic client-side scripting," *MDN Web Docs*, 2026. [Daring]. Tersedia: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript>. [Diakses: 10 Mei 2026].

[2] ECMA International, "ECMAScript 2015 Language Specification," *ECMA-262 6th Edition*, Jun. 2015. [Daring]. Tersedia: <https://262.ecma-international.org/6.0/>. [Diakses: 10 Mei 2026].

[3] Tim Pengajar Praktikum, "BAB X - JavaScript," dalam *Modul Praktikum Pemrograman Web I*, Program Studi Teknologi Rekayasa Perangkat Lunak.